

Instituto Federal de Educação, Ciência e Tecnologia
Curso: Bacharelado em Ciência da Computação
Disciplina: Projeto e Análise de Algoritmos
Professor: Mário Luiz Rodrigues Oliveira
Atividade: Lista Exercícios II
Formiga,MG, 16 de agosto de 2026

Instruções:

1. Essa lista não será avaliada e, portanto, não precisa ser entregue. Resolva os exercícios como forma de fixação do conteúdo ministrado.

2. Caso você ache que falta algum detalhe nas especificações, você deverá fazer as suposições que julgar necessárias e escrevê-las junto à resolução do trabalho. Pode acontecer também que a descrição dessa atividade contenha dados e/ou especificações supérfluas para sua solução. Utilize sua capacidade de julgamento para separar o supérfluo do necessário.

3. Não basta escrever apenas o resultado. É necessário mostrar a resolução ou o raciocínio utilizado para responder às questões. Portanto, justifique todas as suas respostas.

Questão 1. Considere as informações abaixo e responda o que se pede. Justifique suas respostas à luz da teoria da complexidade de algoritmos. Questão inspirada em exercícios análogos propostos em [BRASSARD and BRATLEY, 1996].

(a) Considere os algoritmos A e B com funções de complexidade no pior caso, respectivamente, $\theta(n^3)$ e $\theta(n^2)$. Em alguns conjuntos de dados, o tempo de execução do algoritmo A é menor que o tempo de execução do algoritmo B, quando testados no mesmo ambiente (mesma arquitetura, mesmo sistema operacional, mesma linguagem de programação e mesmo compilador). Forneça três possíveis explicações para esse fenômeno.
(b) Um pesquisador tem a sua disposição 2 algoritmos para resolver um determinado problema. Um algoritmo possui função de complexidade $O(n^2)$ e outro $O(n^3)$. Esse pesquisador escolheu o algoritmo com complexidade $O(n^3)$. Cite 4 razões que poderiam ter motivado essa opção.
(c) Considere os algoritmos A e B para resolver um determinado problema. O algoritmo A exige $10^8 * n$ instruções e o algoritmo B exige $10 * n^2$ instruções. Pode-se afirmar que o algoritmo A é sempre preferível em relação ao algoritmo B?

Questão 2. Considere a teoria da complexidade de algoritmos e responda o que se pede. Justifique formalmente suas respostas. Respostas sem justificativas não serão consideradas.

(a) Explique por que a frase "O tempo de execução do algoritmo A é no mínimo $O(n^2)$"não faz sentido.
(b) Se um algoritmo A tem tempo de execução $\theta(n^2)$ sobre qualquer entrada de tamanho n, ele pode ter tempo de execução $\Omega(nlogn)$ no pior caso?

Questão 3. Prove que as sentenças abaixo são verdadeiras ou falsas, usando para isso as definições das notações assintóticas e/ou contra exemplos. Considere que as funções $f(n)$ e $g(n)$ são assintoticamente não negativas.

(a) $f(n) \in O(g(n))$ se e somente se $g(n) \in \Omega(f(n))$

(b) Se $f(n) \in O(g(n))$ então $g(n) \in O(f(n))$.

(c) Se $f(n) \in O(u(n))$ e $g(n) \in O(v(n))$ então $f(n) + g(n) \in O(u(n) + v(n))$.

(d) Se $f(n) \in O(g(n))$ e $g(n) \in O(f(n))$ então $f(n) \in \theta(g(n))$.

Questão 4. Considere n representando um valor variável e k é um valor constante. Analise os códigos abaixo e encontre a complexidade assintótica em cada caso usando a notação *Big O*. Dê um limite assintótico tão firme quanto possível. Ao realizar a análise, considere o pior caso. Justifique suas respostas. Respostas sem justificativas não serão consideradas.

- **código 1:**

```
soma = 0;
i = 1;
While ( i <= n )
{
    soma = soma + 1;
    i = i * 10;
}
```

- **código 2:**

```
soma = 0;
i = n;
While ( i > 0 )
{
    soma = soma + i;
    i = i / 2; // considere divisao inteira
}
```

- **código 3:**

```
x = escolha (1,n); // retorna um valor inteiro no intervalo [1..n] em O (1)
soma = 0;
for (i = 1; i <= n; i = i + 1)
{
    if ( i != x )
    {
       soma = soma + i;
    }
    else
    {
        for (j = 1; j <= i; j = j + 1)
        {
            soma = soma + j * x;
        }
    }
}
```

- **código 4:**

```
x = escolha (1,n); // retorna um valor inteiro no intervalo [1..n] em O(1)
soma = 0;;
for (i = 1; i <= n; i = i + 1)
{
    if (i == x)
    {
       soma = soma + i;
    }
    else
    {
        for (j = 1; j <= i; j = j + 1)
        {
            soma = soma + j * x;
        }
    }
}
```

- **código 5:**

```
soma = 0;
for (i = 1; i <= n; i = i + 1)
{
    for (j = 1; j <= i; j = 3*j)
    {
        for (k = 1; k <= 25; k = k + 1)
        {
            soma = soma + 1;
        }
    }
}
```

Informações que podem ser úteis

Análise de Algoritmos:

Não há um conjunto de regras para analisar um algoritmo. No entanto, conforme [AHO et al., 1983] podemos seguir o seguinte guia, considerando o modelo computacional *Random Access Machine* (RAM):

1. O custo de execução de uma instrução de atribuição, de leitura ou de escrita é O(1);
2. O custo de execução de uma sequência de instruções é determinado pela instrução de maior custo na sequência de instruções;
3. O custo de execução de um comando de decisão/seleção é composto pelo custo de execução das instruções no corpo do comando de decisão/seleção mais o custo para avaliar a condição. O custo para avaliar a condição é O(1), e
4. O custo de execução de um comando de repetição (laço/loop) é a soma do custo de execução do corpo do comando de repetição adicionado ao custo de avaliar a condição de término do comando de execução, multiplicado pelo número de iterações do comando de repetição. O custo para avaliar a condição de término é O(1).

Somatórios e Logaritmos:

- $\sum_{i=1}^{n}(i) = n(n+1)/2$
- $\sum_{i=1}^{n}(a^i) = a(a^n - 1)/(a-1)$

- $\log(a * b) = \log a + \log b$
- $\log(a/b) = \log a - \log b$
- $\log_b a = \log_c a / \log_c b$
- $\log_b a^c = c * \log_b a$
- $\lg a = \log_2 a$

Hierarquia de funções:

A seguinte hierarquia de funções pode ser definida do ponto de vista assintótico:

$1 \prec log(n) \prec \sqrt{n} \prec n \prec n * log(n) \prec n^c \prec c^n \prec n! \prec n^n$ onde c é uma constante arbitrária inteira ≥ 2.

Referências

[AHO et al., 1983] AHO, A. V., HOPCORFT, J. E., and ULLMAN, J. D. (1983). *Data Structures and Algorithms*. Addison-Wesley.

[BRASSARD and BRATLEY, 1996] BRASSARD, G. and BRATLEY, P. (1996). *Fundamentals of Algorithms*. Prentice-Hall.